



BLOCK VENTURE CHAIN CAPITAL

Agent Wallet Informe de Seguridad y Pruebas

Análisis estático, remediación de vulnerabilidad y batería de pruebas de permisos del agente on-chain

OBJETO	BVCC Agent Wallet (smart account ERC-4337 / ERC-7579)
RED	Arbitrum Sepolia (421614) y Arbitrum One (42161) · Factories V2/V3 también en Ethereum, Base, BNB Chain y Polygon
WALLET PARCHEADA	0x1ED261BA084c4E6117daf289f3ca233BBA3cF0C3
TRABAJO	Pruebas de seguridad internas y verificación de la remediación
FECHAS	2026-06-06 → 2026-07-28 (V1 · V2 · V3 · V4 + revisión externa)
ESTADO	7 HIGH detectados 3 corregidos y desplegados 4 corregidos, pendientes V4 1 ABIERTO 303/303 tests

1 Resumen ejecutivo

Este informe documenta las pruebas de seguridad de la BVCC Agent Wallet — una smart account ERC-4337 que permite al propietario delegar una capacidad de gasto acotada a un agente autónomo (EOA). El trabajo abarcó análisis estático, un hallazgo de severidad alta con su remediación, el redespigie de los contratos parcheados y la re-verificación completa del sistema de permisos del agente sobre el bytecode parcheado. Una segunda ronda (V2) corrigió un problema de gas en mainnet en la lógica de snapshot de comisiones y concluyó con un despliegue multichain determinista (ver §9). Una tercera ronda, V3, reconstruyó la vía DeFi del agente en torno a call policies por selector y un registry de validators on-chain, y se verificó en Arbitrum One mainnet (ver §10–§12).

7 HALLAZGOS ALTOS	303 TESTS, TODOS PASAN	7 ARREGLOS PENDIENTES DE V4	1 HALLAZGO ALTO ABIERTO
-----------------------------	-------------------------------------	--	--------------------------------------

Resultados clave

- **Vulnerabilidad de severidad alta detectada y corregida.** Un bypass de approve permitía al agente mover tokens más allá de sus presupuestos diario/total vía transferFrom. Reproducible on-chain, parcheado en el código y re-verificado tras el redespigie.
- **Problema de gas en mainnet corregido (V2).** Los precompiles de Arbitrum detectados por el escaneo de calldata del snapshot de comisiones quemaban todo el gas reenviado, inflando las estimaciones hasta ~32M y rompiendo los swaps. Cada sonda balanceOf está ahora capada en gas (PROBE_GAS_CAP = 100k); las estimaciones vuelven a ~400k para un swap real (ver §9).
- **Exfiltración de fondos del agente cerrada (V3).** Una clave de agente robada podía enrutar un swap o un withdraw de Aave a su propia dirección sin tocar su presupuesto. V3 hace las llamadas DeFi del agente default-deny por selector y ancla el destinatario a la wallet, o lo valida on-chain mediante un registry de validators cuya gobernanza es inmediata para denegar y con timelock para permitir. Verificado con un test adversarial en mainnet que revierte antes de ejecutar (ver §10–§12).
- **Dos bypasses más cazados antes de publicar.** Una máscara de bytes de comando en el validator del Universal Router habría aceptado un depósito en un puente cross-chain como si fuera un swap; y el increaseLiquidity de Uniswap v3, que no lleva comprobación de propiedad, se excluyó deliberadamente de la whitelist en lugar de anclarlo. Ambos quedan documentados en §10 y §11.
- **Reentrada cruzada — corregida en código, pendiente de desplegar.** Como el control de «el agente es un EOA» solo corría al autorizar, una dirección de agente que adquiriera código después —una delegación EIP-7702 firmada con la propia clave del agente, o un despliegue CREATE2— podía reentrar al execute() de nivel propietario y saltarse todos los límites de golpe. Un PoC drenó 49,925 ETH contra un presupuesto vitalicio de 0,003 ETH. Afecta **igual a V1, V2 y V3**, confirmado drenando una wallet creada desde la factory V2 viva sobre un fork de mainnet. Ahora lo corrigen dos capas, elegidas midiendo tres candidatos contra tres formas de ataque en vez de por argumento (§12.1).
- **Apropiación de guardianes — corregida en código.** Un revisor externo encontró que la dirección de una wallet se deriva solo de la passkey, mientras que sus guardianes los elegía quien llamara primero al createWallet abierto, de forma permanente. Se reprodujo la apropiación

completa: ocupar la dirección, agotar el timelock de recuperación y rotar al propietario. La factory ya no elige guardianes, y fijarlos exige ahora la passkey del dueño (§13.1).

- **Los presupuestos de token no atan el gasto DeFi — abierto, y peor de lo que se escribió al principio.** Donde exista una allowance viva, un agente puede mover un token por una llamada a protocolo sin tocar ningún contador y, como ningún validator inspecciona cantidades ni precios, puede cambiar un activo valioso por uno sin valor a través de un pool que él controla. El anclaje de destinatario manda el output inútil a casa; el valor se queda con el atacante. El plan escalonado está en §13.2.
- **Análisis estático en las tres rondas.** Slither 0.11.3 se pasó antes del despliegue de V1, tras el fix de gas de V2 y otra vez sobre el árbol de V3. La pasada de V1 motivó un control defensivo de dirección cero en los constructores de ambas factories; la de V3 produjo el hallazgo de reentrada más dos falsos positivos (§12).
- **Tres cuestiones más de la revisión externa,** todas reproducidas contra el código: los presupuestos de token no atan el gasto DeFi de caso 3 cuando existe una allowance previa, revocar un agente no retira sus allowances, y approve puede colar ETH saltándose la lista de destinatarios (§13).
- **Después se atacaron directamente la recuperación y la vía de firma** — las dos áreas que solo se habían probado en comportamiento correcto. La firma aguantó los diez ataques, incluidos el replay entre cadenas y entre las dos wallets del propio dueño. La recuperación dio dos arreglos más: no revocaba los agentes, y los guardianes no se podían rotar nunca (§14).
- **Contratos parcheados red desplegados** en Arbitrum Sepolia, con la clave del deployer sin privilegios (el control queda en una dirección de administración independiente).
- **La batería completa de permisos del agente pasa** sobre la wallet parcheada: por transacción, diario, presupuesto total, presupuesto por período (con auto-rollover), expiración, pausa y lista blanca de destinatarios — aplicado tanto para ETH nativo como para tokens ERC-20.

Valoración general. El modelo de permisos del agente es sólido en su diseño y, tras tres rondas de remediación, aplica todas las restricciones configuradas en las vías que comprueba. Hay siete arreglos escritos y probados pero sin desplegar, así que todas las wallets vivas en las seis redes siguen ejecutando el bytecode vulnerable hasta que salga una generación V4 — ese hueco, y no el código, es el riesgo vivo. Dos hallazgos quedan abiertos por decisión, no por descuido: el gasto DeFi de caso 3 no está atado por los presupuestos de token (§13.2), escalonado entre la interfaz y la capa de validators porque no cabe en el bytecode restante de la wallet; y un solo guardián puede bloquear una recuperación (§14.2), lo que deniega el servicio sin mover fondos. Para ambos hay mitigación operativa hoy.

2 Alcance y metodología

En alcance

- `BVCCAgentWallet.sol` — lógica de permisos del agente (`executeAsAgent`, caps por token y ETH, pausa, expiración).
- `BVCCWallet.sol` — smart account base (ejecución, comisiones, recovery por guardianes).
- `BVCCAgentWalletFactory.sol` / `BVCCWalletFactory.sol` — factories CREATE2.
- **Añadidos de V3:** la capa de call policies dentro de `executeAsAgent`, `BVCCValidatorRegistry.sol`, `BVCCUniversalRouterValidator.sol` y la interfaz `IBVCCValidator`, además de los presets de policies que registra el frontend.

Metodología

- **Análisis estático** — Slither 0.11.3 sobre ambos repositorios: antes del redesplicue de V1 (§3), otra vez tras el endurecimiento de gas de V2, y otra vez sobre el árbol de V3 (§12).
- **Pruebas unitarias** — Foundry (`forge test`), con tests de regresión para cada hallazgo.
- **Fuzz y fork (V3)** — el validator del router se fuzzea contra un modelo del router real sobre todo su espacio de comandos; Aave se ejercita sobre un fork de mainnet.
- **Pruebas on-chain** — el EOA del agente ejecuta `executeAsAgent` directamente; los reverts se confirman por simulación (`eth_call`) contra el estado real y los casos válidos como transacciones reales. V3 se ejercitó además en Arbitrum One mainnet, incluida la variante adversarial de un swap real.

Modelo de ejecución del agente

El agente llama a `executeAsAgent(bytes32 mode, bytes executionData)` como una transacción EOA ordinaria (no una `UserOp`). `mode` es el selector de batch ERC-7579 (`0x0100..00`); `executionData` es un `Execution[]` codificado en ABI de (`target`, `value`, `callData`). El orden de validación dentro de la llamada es **per-tx** → **destinatario/protocolo** → **período** → **diario** → **total**, con el estado actualizado **antes** de la ejecución externa (checks-effects-interactions) y un guard `nonReentrant`.

3 Análisis estático — Slither

Slither 0.11.3 se ejecutó sobre ambos repositorios antes del redesplicue. Reportó 39 resultados en 59 contratos; **ninguno accionable**. La tabla resume las categorías y su tratamiento.

CATEGORÍA	EJEMPLOS	TRATAMIENTO
Patrones por diseño	Comparaciones con <code>block.timestamp</code> , ensamblador, llamadas de bajo nivel, calls-en-bucle (batch)	Aceptado
Falsos positivos	Mapping “no inicializado”; reentrancy en <code>createWallet</code> (solo un <code>emit</code> tras un <code>setGuardians</code> de confianza)	Sin acción
Código muerto — <code>_feeNumerator()</code>	Marcado como no usado	Falso positivo — verificado en uso (ver nota)
Falta zero-check — <code>owner_</code> de la <code>factory</code>	El constructor asigna <code>owner</code> sin validar	Corregido
Cosmético	Nomenclatura, dígitos literales, complejidad ciclomática	Aceptado

Nota de verificación — un hallazgo de herramienta no es un hecho. Slither marcó `_feeNumerator()` como código muerto eliminable. Una comprobación del código mostró que **sí** se usa para calcular la comisión (base 0,05% / override del agente 0,15%); eliminarlo habría roto la lógica de comisiones. El hallazgo era un falso positivo causado por el dispatch de funciones virtuales y **no** se actuó sobre él.

Remediación aplicada

Se añadió un control defensivo de dirección cero — `if (owner_ == address(0)) revert ZeroOwner();` — al constructor de ambas `factories`, sincronizado en los dos repositorios. Suite completa en verde: **forge test = 128/128** en cada repo. Slither se re-ejecutó sobre los contratos V2 tras el endurecimiento de gas (§9): **0 resultados**; la suite unitaria queda en **131/131**.

4 Hallazgo — Bypass de caps de token vía `approve`

ALTA

CORREGIDO Y REDESPLEGADO

Descripción

Los presupuestos **diario** y **total** por token del agente podían saltarse por completo. La validación contabilizaba el gasto de `transfer`, pero cargaba `approve` solo contra el cap por transacción — no contra el diario ni el total, y sin restringir el spender. El agente podía por tanto `approve` un allowance (solo capeado por per-tx) y vaciarlo con un `transferFrom` externo, sin tocar nunca los contadores acumulados.

Prueba de concepto (ejecutada on-chain, presupuestos agotados a diario 2/2 · total 2/3)

PASO	ACCIÓN	RESULTADO
1	<code>executeAsAgent → USDC.approve(B, 1)</code>	Pasa — per-tx OK, diario ignorado
2	Como EOA: <code>USDC.transferFrom(wallet, B, 1)</code> (fuera de <code>executeAsAgent</code>)	1 USDC extraído; contadores sin cambios

Un transfer plano del mismo importe sí revirtió correctamente con `TokenDailyLimitExceeded`; `approve` no. Al no tener límite diario, el ciclo `approve(1) → transferFrom(1)` era repetible — vaciando la wallet más allá de los caps diario (2) y total (3). Los fondos de la PoC se devolvieron a la wallet.

Remediación

En el bucle de acumulación de `executeAsAgent` se extendió la condición con `|| _isApprove(batch[i].callData)` de modo que `_applyTokenSpend` carga los `approve` contra los mismos caps diario/total y actualiza los contadores. La elección es deliberadamente conservadora (un `approve` sobrescribe el `allowance`, por lo que puede sobre-contar — la dirección segura). Se añadieron seis tests de regresión; **5 fallan sin el fix, todos pasan con él**.

Superficie acotada. Los tokens aceptan solo `transfer` y `approve` por selector; `transferFrom` e `increaseAllowance` están bloqueados. ETH no tiene primitiva de delegación, así que no se ve afectado. `approve` era el único vector — ahora cerrado y re-verificado tras el redesplicue (ver §6).

5 Despliegue de la remediación

Ambas factories cambiaron de bytecode (zero-check), produciendo nuevas direcciones deterministas CREATE2 en Arbitrum Sepolia. La configuración de red del frontend se actualizó en consecuencia.

CONTRATO	NUEVA DIRECCIÓN (PARCHEADA)	ANTERIOR (DEPRECADA)
AgentWalletFactory	0xc87aa10747A92B472EF6B36e190B84c897a2953e	0xe7ad...7e04
SmartWalletFactory	0xa5290A51a73903176e09C864E1542a07da67BD12	0x6890...F58d

Separación de privilegios

- **Deployer (sin control):** 0xA3e06B6443D911915bb4eD157832d43C25D6617D — el owner de la factory se fija desde un argumento del constructor, no desde `msg.sender`, así que el deployer no tiene privilegios.
- **Admin / kill-switch:** 0x3145Bd5e2489d8bDdAb17F23F26F07Ac5aD55A4c — inmutable; la única acción privilegiada es el `kill()` de un solo sentido.

Tamaños runtime dentro del límite EIP-170 (AgentWallet 21.169 B; AgentWalletFactory 23.803 B — 773 B de margen).

6 Batería de pruebas de permisos del agente

Ejecutada contra la wallet parcheada 0x1ED261...F0C3 con el EOA del agente 0x3852...4aB4. Reverts confirmados por simulación; casos válidos como transacciones reales. Los destinatarios permitidos fueron los dos guardianes de recovery (controlados por el bot, fondos recuperables).

#	PRUEBA	ESPERADO	RESULTADO
1	ETH per-tx (0,0002 > máx 0,0001)	revierte	ExceedsPerTxLimit
2	ETH dentro de límite (0,0001)	pasa	Success
3	USDC per-tx (2 > máx 1)	revierte	ExceedsTokenMaxAmount
4	USDC diario (1+1+1, cap 2) — txs reales	la 3ª revierte	TokenDailyLimitExceeded
5	Envío de token a destino no permitido	revierte	RecipientNotAllowed
6	Envío de ETH a destino no permitido	revierte	RecipientNotAllowed
7	Presupuesto total (0,0002+0,0001 = 0,0003) — txs reales	la siguiente revierte	AgentBudgetExceeded
8	Presupuesto período (0,0003+0,0002 = 0,0005) — txs reales	la siguiente revierte	PeriodBudgetExceeded
9	Auto-rollover del período (periodStart 0)	la 1ª tx resetea la ventana	Verificado
10	Pausado (el owner pausa)	agente bloqueado	EnforcedPause
11	Expiración (caducó sola)	agente bloqueado	AgentPermissionsExpired

Observación. La lista blanca de destinatarios aplica también a los **transfers de token**, no solo a ETH — un envío de token a una dirección no permitida revierte `RecipientNotAllowed`. La re-

autorización vía "Edit" no resetea el contador diario (indexado por día UTC); con `periodStart = 0` la primera operación rota la ventana del período.

Resistencia a escalada de privilegios (de la sesión del 06-06)

Un agente que apunta a la propia wallet para llamar a funciones de owner (`pauseAgents`, `revokeAgent`, `authorizeAgent`) revierte con `AgentCannotCallWallet`; las llamadas arbitrarias a contratos fuera de la lista de protocolos están bloqueadas. El agente solo puede mover ETH y tokens permitidos hacia destinatarios permitidos.

7 Correcciones de soporte (frontend)

Se corrigieron tres incidencias en la aplicación web de la wallet, sincronizadas con el repositorio público.

ÁREA	INCIDENCIA	CORRECCIÓN
lib/wallet.ts	La recuperación de credencial consultaba solo el evento de la factory base, por lo que reabrir una wallet <i>de agente</i> por dirección fallaba (“no hay wallet activa”).	Consultar también agentFactory / AgentWalletCreated.
Página de agentes	Los límites por token se mostraban solo como texto.	Leer getTokenSpent on-chain; pintar barras de progreso Diario/Total (verificado en vivo).
Flujo de deploy	El gas que pasaba la app infraestimaba el deploy de la agent wallet (~21 KB) en Arbitrum, provocando fallos al firmar.	Delegar en la estimación de red de la wallet.

8 Recuperación por Guardianes — Ejecutada y Verificada

La wallet base admite recuperación social: un quórum de 2 de 3 guardianes puede rotar la passkey del propietario tras un timelock. Se probó de extremo a extremo en la wallet 0xEBc851bA9e8Ce32A2cb8eB99544F5F024f06E61d y ya está **completada**.

Proceso

- **Inicio (2026-06-06).** Recovery propuesta hacia una nueva passkey de propietario; los guardianes 1 y 2 (0xa337...5908, 0x8e06...2Ccc) aprobaron — quórum 2/2.
- **Timelock.** recoveryReadyAt on-chain = 2026-06-08 17:24 UTC; durante toda la ventana el propietario conservó la capacidad de cancelar.
- **Ejecución (2026-06-08).** Transcurrido el timelock se llamó executeRecovery() desde la UI /recover. El signer on-chain rotó a la nueva passkey (X 0x303591... / Y 0xb858ed...), reemplazando al propietario anterior (X 0x070f7a...).
- **Comprobación positiva.** El nuevo propietario firmó de inmediato una transferencia real de **19 USDC** con el nuevo Face ID — confirmada on-chain.

Prueba negativa — la passkey antigua queda anulada

Para confirmar que la rotación revocó realmente la clave anterior, se intentó una transferencia con la passkey **antigua**, cuya credencial WebAuthn sigue presente en el autenticador de pruebas. La firma se genera localmente, pero debe validarse on-chain contra el propietario actual.

#	ACCIÓN	ESPERADO	RESULTADO
1	Enviar 0.0001 ETH, UserOp firmado por la passkey ANTIGUA	rechazo en validación	FailedOp(0, AA24 signature error)
2	Efecto on-chain (saldos ETH / USDC y nonce)	sin cambios	Sin cambios

Resultado. La recovery se completó con éxito: la propiedad pasó a la nueva passkey, la nueva clave opera la wallet y la antigua es rechazada criptográficamente por el EntryPoint durante la validación de firma — ya no puede mover fondos aunque su credencial local siga existiendo.

9 V2 — Endurecimiento de gas en mainnet y despliegue multichain

ALTA (disponibilidad / sobrepago)

CORREGIDO — V2 DESPLEGADA 2026-06-11

Descripción

En Arbitrum mainnet, los swaps a través de la wallet fallaban en la estimación de gas. La lógica de comisiones del Caso 3 escanea el calldata buscando candidatos a token y sondea cada uno con `balanceOf`. Los precompiles del sistema de Arbitrum (`0x64-0x6f`) reportan bytecode (`0xfe`), por lo que el control por longitud de código no los filtraba — y llamarlos consume **todo el gas reenviado** en lugar de revertir barato. El estimador subía el límite de gas hasta ~32M para que la sonda “cupiera”; ese límite inflado es lo que la wallet habría prefinanciado (riesgo de sobrepago 64×) y, en la práctica, la operación fallaba.

	ANTES (BUG)	DESPUÉS (FIX V2)
Estimación de gas (frontend / bundler)	~32.000.000	~400.000
Efecto de un candidato quema-gas	quema todo el gas reenviado	capado a 100k, la ejecución continúa
Resultado	el swap falla (o sobrepago 64×)	swap normal, ~\$0,02–0,05 en Arbitrum

Remediación (V2)

- **Sondas capadas en gas.** Una constante compartida `PROBE_GAS_CAP = 100_000` capta cada `staticcall` de `balanceOf` tanto en `_tryBalanceOf` (snapshot) como en `_collectFeesOnIncrease` (cobro de comisión). El peor caso de un escaneo completo queda acotado a ~2,2M de gas frente a los 32M anteriores; un ERC-20 real necesita mucho menos que el cap, así que la contabilidad de comisiones no se ve afectada.
- **El cobro de comisión nunca puede romper una llamada del usuario.** Si un token se comporta mal tras el swap, su comisión se omite en lugar de revertir la transacción del usuario.
- **Cota contable explícita.** `executeAsAgent` ahora exige `totalEth ≤ type(uint128).max` antes del cast, eliminando un caso límite de truncamiento silencioso para agentes configurados sin límites de ETH.

Verificación

- **forge test = 131/131** — tests de regresión añadidos: un candidato quema-gas no deja sin gas al swap; un `balanceOf` que revierte se filtra sin ser fatal; un token con retorno sobredimensionado se snapshotea igualmente.
- **Slither 0.11.3 re-ejecutado sobre V2: 0 resultados.** (Nota de herramienta: Slither no puede generar IR para los internos del `ReentrancyGuard` con `transient storage` de OpenZeppelin — limitación conocida del parser, no un hallazgo.)
- **Tamaños EIP-170:** AgentWallet 21.214 B; AgentWalletFactory 23.848 B (728 B de margen); SmartWallet 13.432 B; SmartWalletFactory 16.030 B.

Despliegue V2 — CREATE2 determinista, misma dirección en todas las redes

CONTRATO	DIRECCIÓN (IDÉNTICA EN TODAS LAS REDES)
----------	---

SmartWalletFactory V2	0x230b7010529AB6977Dd8581B3eF018ef865BdEf1
AgentWalletFactory V2	0x8D9e24022777173AD6336e00884b6C87c7EF054c

Desplegadas el 2026-06-11 en Arbitrum One (42161), BNB Chain (56) y Arbitrum Sepolia (421614). Las wallets creadas por factories pre-V2 no reciben el fix — las wallets nuevas deben crearse a través de las factories V2.

10 V3 — Modelo de seguridad por call policies

ALTA (exfiltración de fondos por el agente)

CORREGIDO — V3 DESPLEGADO 2026-07

Descripción

Hasta V2, el presupuesto del agente se descontaba cuando el valor salía de la wallet por una vía que la wallet podía ver: una transferencia nativa, un transfer ERC-20 o un approve. Una llamada a protocolo (caso 3) se permitía en cuanto su destino estaba en `allowedProtocols`. Eso no basta, porque varios puntos de entrada DeFi reciben el destino como argumento:

- `Pool.withdraw(asset, amount, to)` de Aave envía la posición suplida directamente a `to` — sin `approve` y sin `transfer`, así que el presupuesto no se enteraba.
- Las llamadas de swap y LP (`exactInputSingle`, `execute` del Universal Router, `mint`) llevan un argumento `recipient` que el agente elegía libremente.

Una clave de agente robada o defectuosa podía por tanto poner su propia dirección como destino y vaciar todo lo que la wallet hubiera suplido o aprobado a un protocolo whitelisteado **sin consumir un solo wei de su presupuesto**. La severidad es ALTA: la pérdida solo está acotada por la exposición DeFi de la wallet, y la contabilidad on-chain no muestra nada.

Remediación (V3) — call policies por selector

Las llamadas de caso 3 son ahora **default-deny por selector**. Whitelistar el protocolo ya no basta: el propietario debe registrar además una policy que diga *a dónde puede ir el dinero*. Las policies se escriben con `setCallPolicy(target, selector, policy)`, que solo puede llamar la propia wallet — en la práctica el passkey del propietario, incluido en la misma firma que autoriza al agente. La policy es una única palabra `uint256`:

BITS	CAMPO	SIGNIFICADO
255	POLICY_ALLOWED	El selector está permitido. Sin él → <code>SelectorNotAllowed</code>
254	POLICY_DEEP	Deriva la llamada al registry de validators on-chain (ver §11)
223..192	PIN_WALLET (bitmap 32 bits)	La palabra <i>i</i> del calldata debe ser <code>address(this)</code>
191..160	PIN_PROTOCOL (bitmap 32 bits)	La palabra <i>i</i> debe ser una dirección de <code>allowedProtocols</code>

La comprobación dentro de `executeAsAgent` ocurre antes de cualquier llamada externa: (1) se lee el selector del calldata (`< 4 bytes` → `CalldataTooShort`); (2) la policy debe llevar `POLICY_ALLOWED`; (3) cada palabra anclada se compara **a palabra completa**, lo que además rechaza bits altos sucios, y una palabra leída más allá del final del calldata vale cero y por tanto falla — la comprobación es `fail-closed` por construcción; (4) si está `POLICY_DEEP`, el registry debe devolver `true`. Se añadieron cuatro custom errors: `SelectorNotAllowed`, `PinnedArgMismatch`, `CalldataTooShort` y `PolicyValidationFailed`. También cambió un default relacionado: una lista `allowedProtocols` vacía ahora revierte `NoProtocolsWhitelisted` en lugar de significar «cualquier protocolo».

El propietario nunca está sujeto a las call policies. Solo aplican a los agentes. Una persona que firma con su passkey sigue usando cualquier dApp con normalidad — V3 estrecha la vía delegada, no la del propietario.

Policías registradas (se muestra Arbitrum One; las otras cinco redes usan las mismas formas)

PROTOCOLO	SELECTORES	ANCLAJE
Uniswap SwapRouter02	<code>exactInputSingle · exactOutputSingle</code>	PIN_WALLET palabra 3
Uniswap SwapRouter02	<code>exactInput · exactOutput (multi-hop)</code>	PIN_WALLET palabra 2
Aave v3 Pool	<code>supply · withdraw · borrow · repay</code>	PIN_WALLET palabra 2 / 2 / 4 / 3
Aave v3 Pool	<code>repayWithATokens · setUserUseReserveAsCollateral · setUserEMode</code>	Permitido sin anclaje — no existe argumento de destino
Permit2	<code>approve(address, address, uint160, uint48)</code>	PIN_PROTOCOL palabra 1 (el spender)
Uniswap Universal Router	<code>execute(bytes, bytes[], uint256)</code>	DEEP → registry de validators
Uniswap v3 NFPM	<code>mint · collect</code>	PIN_WALLET palabra 9 / 1
Uniswap v3 NFPM	<code>decreaseLiquidity · burn</code>	Permitido sin anclaje — el NFPM los limita al propietario del tokenId
WETH	<code>deposit · withdraw</code>	Permitido sin anclaje — ambos acreditan/pagan a <code>msg.sender</code>

Dos trampas de calldata encontradas al escribir los presets.

- **Desplazamiento del offset.** El struct de `exactInput` (multi-hop) lleva un `bytes path`, lo que vuelve dinámica la tupla y mete una palabra de offset por delante: el destinatario está en la palabra 2, no en la 3 como en las variantes `*Single`. Anclar ahí la palabra 3 habría anclado `amountIn` y habría dejado el destino libre. Todos los offsets se re-derivaron codificando calldata real.
- **`increaseLiquidity` queda deliberadamente fuera de la whitelist.** A diferencia de `decreaseLiquidity`, `collect` y `burn`, el NFPM de v3 no le aplica ninguna comprobación de propiedad — solo el deadline. Un agente habría podido empujar los tokens que la wallet tiene aprobados al NFPM hacia un `tokenId` propiedad del atacante: un bypass de la lista de destinatarios que ningún anclaje de bitmap puede expresar, porque la palabra 0 es un `tokenId`, no una dirección. Los agentes añaden liquidez minteando una posición nueva con el destinatario anclado a la wallet; reactivar `increaseLiquidity` exigiría un validator profundo que compruebe `ownerOf(tokenId) == wallet`.

Qué cubre cada capa

CASO	OPERACIÓN	GOBERNADO POR
1	Transferencia de ETH nativo	<code>allowedRecipients</code> + límites nativos
2	<code>transfer(to, amount)</code>	<code>allowedRecipients</code> + límites de token
2b	<code>approve(spender, amount)</code>	<code>allowedRecipients</code> + límites de token

Nota de alcance. V3 cierra la exfiltración de fondos por swaps, LP y retiradas de protocolo: el caso 3 es ahora seguro por construcción, configure lo que configure el propietario. Lo que no hace es convertir a un agente comprometido en inofensivo — un `transfer` o un `approve` directos (casos 2 y 2b) siguen gobernados por la lista de destinatarios y los límites por token, y esos los elige el propietario. Simulado en mainnet contra el agente de pruebas, con la lista de destinatarios vacía y los límites a cero, un `usdc.transfer` directo a un atacante pasa; con destinatarios y límites configurados revierte `RecipientNotAllowed`. Los límites on-chain son el modelo de seguridad.

11 Registry de validators y validación profunda

Por qué hace falta un segundo mecanismo

Un anclaje de bitmap solo alcanza una palabra en un offset fijo. En el `execute(bytes commands, bytes[] inputs, uint256 deadline)` del Universal Router — y en el `modifyLiquidities` de Uniswap v4 — el destinatario va enterrado dentro de datos de longitud variable, así que no hay offset fijo. Esos selectores llevan en su lugar el bit DEEP, que delega la decisión en un validator on-chain sin estado.

Diseño

- **Dirección de despacho fija.** `BVCCValidatorRegistry` está compilado dentro de la wallet como constante (`0x5e371D54AC97a57B0a99145Ed04A3c9fA07850C2`). Como la palabra de policy solo lleva un flag y nunca una dirección, a un propietario no se le puede colar el validator de un atacante. Un test dedicado (`Create2Consistency.t.sol`) rompe la build si esa constante llega a desviarse de la dirección `CREATE2` del registry.
- **Despacho fail-closed.** Sin validator registrado para el destino → `false`. Un validator que revierte, o que no tiene código, también deniega. Solo un `true` explícito deja pasar la llamada; cualquier otra cosa revierte `PolicyValidationFailed`.
- **Validators sin estado.** Son contratos `view` a los que se llega por `staticcall`. Nunca custodian fondos, nunca reciben `approvals` y no pueden mutar estado — solo responden sí o no.
- **Doble consentimiento.** Habilitar un protocolo complejo exige dos pasos independientes: BVCC registra el validator en el registry y el propietario registra la policy `ALLOWED|DEEP` con su `passkey`. Si falta cualquiera de los dos, la llamada se deniega.

Gobernanza asimétrica

DIRECCIÓN	FUNCIÓN	DEMORA
Permitir — registrar o sustituir un validator	<code>proposeValidator</code> → <code>activateValidator</code>	Timelock de 48 h + evento <code>ValidatorProposed</code>
Denegar — cancelar una propuesta pendiente	<code>cancelProposal</code>	Inmediata
Denegar — desactivar un validator activo	<code>freezeValidator</code>	Inmediata

La demora solo corre en la dirección permisiva, así que una propuesta maliciosa o equivocada es pública durante dos días antes de poder surtir efecto y los propietarios pueden reaccionar (pausar o revocar sus agentes). El peor caso con la clave de administración robada es por tanto una denegación de servicio o —tras 48 horas visibles públicamente— una degradación al nivel de protección de V2 en un protocolo. Nunca es una vía directa a los fondos: el registry no mueve nada.

El validator del Universal Router

- **Atado a un único router.** La dirección del router es un argumento inmutable del constructor; una llamada con cualquier otro destino devuelve `false`. Un router nuevo necesita su propio validator y su propio ciclo de gobernanza de 48 horas.
- **Coincidencia exacta del byte de comando** sobre `{0x00 V3_SWAP_EXACT_IN, 0x01 V3_SWAP_EXACT_OUT, 0x10 V4_SWAP}`. Cualquier otro byte se deniega, incluidas las variantes `allow-`

revert y los comandos desconocidos.

- **Se comprueba cada destinatario**, incluida la acción TAKE de v4 ({0x07 SWAP, 0x0b SETTLE, 0x0e TAKE}): debe ser la wallet o el centinela MSG_SENDER. SWEEP, TRANSFER, PAY_PORTION, WRAP y UNWRAP se deniegan de plano.
- **El input malformado deniega**. Un calldata indecodificable revierte dentro del validator, y el registry lo trata como denegación; una cadena commands vacía se rechaza explícitamente.
- **Alcance**. Swaps token → token. Las patas nativas que dependen del ADDRESS_THIS del router quedan deliberadamente fuera; los swaps nativos se ejecutan como batches propios de BVCC a través de WETH, de modo que el router nunca retiene fondos de la wallet a mitad de camino.

Bug crítico detectado en revisión — la máscara de comandos. La primera versión del validator enmascaraba los bytes de comando con & 0x3f. El router real enmascara con & 0x7f (el bit 0x80 es FLAG_ALLOW_REVERT), lo que significa que 0x40 **no** es un alias de 0x00: es ACROSS_V4_DEPOSIT_V3, un depósito en un puente cross-chain. Con la máscara equivocada el validator lo habría aceptado como un swap v3 mientras el router bridgeaba los fondos de la wallet hacia un atacante en otra cadena: exactamente la clase de exfiltración que V3 existe para impedir. La corrección sustituyó la máscara por coincidencia exacta, ató el validator a un único router y rechazó las cadenas de comandos vacías. El comportamiento se fuzzea ahora sobre los 256 bytes de comando contra un modelo del router, y se volvió a confirmar en mainnet tras el despliegue (ver §12).

Aave no necesita validator

Los argumentos de destino de Aave (onBehalfOf, to) están en offsets fijos, así que se resuelven con anclajes de bitmap dentro de la propia wallet. Ninguna policy de Aave lleva el bit DEEP, el registry no se consulta nunca para Aave y no interviene ningún timelock. Solo necesitarían validator las operaciones cuya seguridad no se puede expresar como «la palabra *N* debe ser la wallet» — flash loans, multicall, rutas por adaptadores — y esas quedan fuera de alcance.

12 V3 — Verificación y despliegue

Batería de pruebas — forge test: 264 pasan, 0 fallan

SUITE	TESTS	FOCO
BVCCAgentWallet.t.sol	50	Núcleo de permisos del agente: límites, presupuestos, expiración, pausa, batches
UniversalRouterValidator.t.sol	42	Corpus del router, destinatarios atacantes, fuzz sobre los 256 bytes de comando
BVCCAgentWalletAdvanced.t.sol	40	Contabilidad de comisiones, regresiones de approve, casos límite
CallPolicyAdversarial.t.sol	34	Intentos de bypass de policies, gobernanza del registry, calldata malformado
BVCCPositionManagerValidator.t.sol	26	Validación del calldata de LP en Uniswap v4
BVCCWallet.t.sol	23	Smart account base: ejecución, comisiones, recuperación por guardianes
BVCCWalletFactory.t.sol · BVCCAgentWalletFactory.t.sol	18	Factories CREATE2, direcciones deterministas, kill switch
BVCCHookRegistry.t.sol	11	Registry de aprobación de hooks y su timelock
AaveIntegration.t.sol	10	Ciclo de Aave sobre fork de mainnet y variantes adversariales
BVCCPMValidatorSdkVectors.t.sol · ERC7739*.t.sol · Create2Consistency.t.sol · GasBench.t.sol	10	Vectores dorados del SDK, firmas ERC-7739, guarda de congelación de direcciones, gas

Tests adversariales de policies (CallPolicyAdversarial.t.sol): los withdraw/supply/borrow/repay de Aave con un to u onBehalfOf externo revierten PinnedArgMismatch; un selector no registrado revierte SelectorNotAllowed; PIN_PROTOCOL rechaza un spender no whitelisteado; una llamada DEEP deniega cuando el validator devuelve false, revierte, no tiene código o no está registrado, y solo pasa con true; el timelock de 48 h, el freeze y el cancel se comportan según lo especificado; policy = 0 revoca; el calldata corto, los anclajes fuera de rango y los bits altos sucios se rechazan; y un batch con un solo elemento malo revierte entero.

Aave sobre fork de mainnet (AaveIntegration.t.sol): ciclo completo supply → borrow → repay → withdraw, con la comisión de agente del 0,15% cobrada exactamente en withdraw y borrow y no cobrada en supply ni repay; consumo de presupuesto verificado por la vía del approve; un beneficiario externo en cualquiera de las cuatro llamadas revierte; flashLoanSimple y un setUserEMode no registrado revierten; un Pool ausente de allowedRecipients revierte; y el batch approve+supply hace rollback atómico si falla.

Análisis estático — Slither re-ejecutado sobre V3

Slither 0.11.3 se volvió a pasar sobre el árbol V3 el 2026-07-27 (slither . --exclude-dependencies, 65 contratos, 100 detectores): **54 resultados — 3 High, 29 Low, 22 Informational**. Cada resultado de impacto alto se contrastó contra el código fuente antes de aceptarlo o descartarlo. Uno de ellos es real: el resultado reentrancy-eth es un hallazgo ALTO confirmado, reproducido con un test de prueba de concepto y todavía abierto a fecha de redacción (§12.1).

La pasada que cuenta es esta, no la de V2. La de V2 devolvió «0 resultados», pero solo porque el analizador no pudo bajar a IR el ReentrancyGuard con transient storage de OpenZeppelin y abandonó (§3). Aquel cero era la herramienta rindiéndose, no un aprobado. El árbol de V3 se analiza de principio a fin —los detectores sí llegan a executeAsAgent, como demuestra el resultado de reentrancy de abajo—, así que este es el dato de análisis estático que tiene peso, y sus clases de resultados encajan con los 39 de la pasada de V1.

DETECTOR	OBJETIVO	VEREDICTO
reentrancy-eth confianza: media	executeAsAgent escribe _currentAgent = 0 después de super.execute()	CONFIRMADO — ALTA — reproducido con un test PoC (§12.1)
uninitialized- state x2 confianza: alta	_tokenTotalSpent, _tokenDailySpent	Falso positivo — ambos son mapping, que no tienen inicializador y valen cero por defecto

12.1 Hallazgo — reentrada cruzada vía _currentAgent

ALTA **CORREGIDO EN CÓDIGO** **PENDIENTE DE DESPLEGAR EN V4**

Este resultado se descartó en un primer momento alegando que un agente es un EOA y que no se le puede ceder el control a mitad de transacción. Ese razonamiento no se sostiene, y el hallazgo queda confirmado.

Mecanismo

executeAsAgent fija _currentAgent = agent alrededor de su llamada a super.execute(), y _erc7821AuthorizedExecutor concede la vía execute() de nivel propietario a cualquier llamante que cumpla caller == _currentAgent. El execute() padre no lleva guard de reentrada — nonReentrant está solo en executeAsAgent— y no aplica ningún límite de agente, porque las comprobaciones viven en executeAsAgent. La premisa del EOA descansa en AgentMustBeEOA (agent.code.length == 0), que se exigía **solo en el momento de autorizar** y nunca se volvía a comprobar al ejecutar. Una dirección puede adquirir código después: mediante una delegación EIP-7702, firmada con la propia clave del agente —justo la clave que el modelo de amenaza asume que un atacante puede tener— o desplegando en una dirección CREATE2 precalculada que estaba sin código cuando se autorizó.

Alcance: afecta igual a V1, V2 y V3. No es una regresión de V3. Los cuatro ingredientes —la bandera, el override del executor, el control de EOA solo al autorizar y el execute() padre sin guard— están literalmente en las tres generaciones; lo único que se movió son los números de línea. Se confirmó contra código desplegado, no contra el fuente: sobre un fork de Arbitrum One, una wallet creada desde la **factory V2 viva** (0x8D9e2402...) fue drenada con el mismo ataque (test/LegacyV2ReentrancyFork.t.sol).

Reproducción

test/AgentReentrancyPoC.t.sol. Se autoriza un agente sin código con topes de 0,001 / 0,002 / 0,003 ETH (por tx / diario / vitalicio). Después se coloca código en la dirección del agente, modelando la delegación. El agente manda un batch cuyo único elemento es una transferencia de **1 wei** a sí mismo —dentro de todos los topes— que le cede el control; desde ahí llama a `execute()` de vuelta y saca **49,925 ETH** de un saldo de 100 ETH. La diferencia de 0,075 ETH es la comisión propia del 0,15% de la wallet, lo que confirma que el drenaje viajó por la vía de propietario.

Radio de impacto

Se saltan de golpe todas las restricciones del agente: por transacción, diaria, de período y vitalicia, la lista blanca de destinatarios y las call policies de V3. Convierte un agente acotado en control equivalente al del propietario, un agujero estrictamente mayor que el vector de exfiltración que V3 vino a cerrar. No afecta a la vía biométrica del propietario, ni permite nada a un tercero que no tenga la clave del agente.

Remediación — dos capas

Se implementaron tres candidatos y se midieron contra tres formas de ataque, en vez de discutirlos. S1 manda 1 wei directamente al agente; S2 deja al agente sin código al entrar y hace que el propio batch le despliegue código a esa dirección mediante un ayudante whitelisteado (sin EIP-7702 de por medio); S3 no nombra al agente en ningún momento y le hace llegar valor a través de un intermediario permitido.

CANDIDATO	S1 DIRECTO	S2 CREATE2	S3 INDIRECTO	TAMAÑO FACTORY
Ninguno (antes del fix)	drena	drena	drena	24 . 283
B — re-chequear <code>code.length</code>	bloquea	drena	bloquea	24 . 294
C — el batch no puede apuntar al agente	bloquea	bloquea	drena	24 . 312
A — guard en el <code>execute()</code> externo	bloquea	bloquea	bloquea	24 . 317
A + B (lo que se publica)	bloquea	bloquea	bloquea	24 . 328

Los dos candidatos descartados fallan por motivos instructivos. **B por sí sola** es otra comprobación puntual —la misma clase de suposición que causó el bug— y S2 se le cuela haciendo que el código aparezca *durante* el batch. **C por sí sola** cierra únicamente la puerta que usó la primera prueba de concepto; S3 llega al agente a través de un protocolo legítimo.

Lo que se publica son las dos capas. La **capa 1** vuelve a comprobar el invariante del EOA en cada ejecución, de modo que se exige de forma continua y no una sola vez. La **capa 2** sobrescribe `execute()` en la wallet de agente para rechazar cualquier entrada externa mientras hay un batch de agente en vuelo (`ReentrantAgentExecute`); `executeAsAgent` llega al padre por `super.execute()`, que resuelve estáticamente y por tanto salta el override, así que el camino legítimo queda intacto. La capa 2 es la que no depende de *cómo* ni de *cuándo* consiguió código el agente: cierra la ventana en sí. Cada capa tiene su propio test de regresión, y la capa 2 queda probada de forma independiente con la forma S2, donde la capa 1 pasa limpiamente y el drenaje se detiene igual.

Coste: la factory de agentes crece 45 bytes hasta 24.328, dejando 248 de margen bajo EIP-170, y `executeAsAgent` en caso 3 sube de 64.758 a 64.903 gas —145 gas, un 0,2%, porque la dirección del agente ya está caliente al ser quien firma la transacción—. Una consecuencia deliberada de comportamiento: un agente que adopte una delegación EIP-7702 por motivos ajenos dejará de funcionar hasta que la quite. Es el invariante que el propio contrato declara, ahora exigido.

Todavía sin desplegar. El arreglo cambia el bytecode de la wallet y por tanto las direcciones CREATE2; Create2Consistency.t.sol falla a propósito como prueba mecánica. Todas las wallets vivas en las seis redes siguen ejecutando bytecode vulnerable hasta que se publique una generación V4 de factories. **Mitigación disponible hoy, sin redesplegar:** autorizar a los agentes con una allowedRecipients no vacía que no contenga la dirección del propio agente, lo que le quita al batch la capacidad de devolverle el control (test_RecipientWhitelist_BlocksTheEntry). Las wallets que se queden en el default permisivo —lista de destinatarios vacía, es decir, cualquier destino— son la configuración expuesta. Pausar o revocar al agente también lo cierra de inmediato.

Entre los Low, 20 calls-loop y 4 timestamp son inherentes a la ejecución por batches y a los límites indexados por día; el return-bomb de _tryBalanceOf está acotado por el mismo PROBE_GAS_CAP que introdujo V2, que también limita cuántos datos de retorno puede permitirse producir un token hostil. Un resultado sí es una carencia real, aunque inerte:

Detalle aceptado — sin zero-check en los constructores de los validators. Ni BVCCUniversalRouterValidator ni BVCCPositionManagerValidator rechazan una dirección cero para el protocolo al que se atan, a diferencia de los constructores de las factories endurecidos en V1 (zeroOwner). Un validator atado a la dirección cero denegaría todas las llamadas —fail-closed, así que el comportamiento desplegado es seguro— y las seis instancias desplegadas llevan los argumentos de constructor correctos, registrados con sus codehashes en contracts/deployments/ y verificados en los exploradores. Añadir el control cambiaría el bytecode, y por tanto las direcciones CREATE2, forzando un redespliegue y un ciclo de gobernanza de 48 horas por red. Queda anotado aquí para la próxima revisión de bytecode, no ejecutado ahora.

Re-ejecución tras los arreglos de V4: 53 resultados — 3 High, 27 Low, 23 Informational. Los tres High no cambian, y los tres son ya falsos positivos: los dos uninitialized-state son mappings, y reentrancy-eth sigue viendo la bandera escrita tras la llamada externa aunque el §12.1 cerrara la ventana — ahora incluso lista execute() entre las funciones alcanzables, que es precisamente donde está el guard. Apareció un segundo dead-code por _afterRecovery(), el hook de recuperación del §14.2: el mismo falso positivo que _feeNumerator(), una función virtual vacía que sobrescribe el hijo. Ninguna de las dos se toca. Los arreglos no introdujeron nada nuevo. **El análisis estático detecta patrones, no emite un veredicto:** esta cifra no se movió mientras la reentrada fue explotable durante tres generaciones, y tampoco se movió cuando el arreglo la cerró.

Los resultados Informational son de las mismas clases que en las pasadas de V1 y V2: ensamblador inline, low-level calls, convenciones de nombres en los inmutables, dispersión de pragmas por el árbol de OpenZeppelin, y el aviso de "dead code" sobre _feeNumerator() — falso positivo conocido: se alcanza por dispatch virtual y fija la comisión de agente del 0,15%.

Verificación on-chain — Arbitrum One (mainnet)

Wallet de pruebas 0x605A4C65dBa64bd26d9F7701e1e4Cda48c594A5c, agente 0x38529C66F3cf22453D66B9E2A20FdF2676544aB4.

FECHA	OPERACIÓN	RESULTADO
2026-07-16	Swap del agente USDC → WETH por SwapRouter02, dirigido por el servidor MCP 0x4768603d38b03f9823d5d25e53fb0397 0b8e4d294ebee791d0ae9ef92e71a8ae	OK USDC debitado exacto, WETH a la wallet, comisión 0,15% exacta

2026-07-20	Swap del agente por el Universal Router — batch atómico de 3 llamadas que atraviesa tres capas distintas 0x6f831b37ae5db240256bf6bf99c8ce418edb75a1c38e37d6ed3e2c3a052a3bc0	OK gas 419.124 · USDC -0,100000 · comisión 0,000000078468831120 WETH = 0,15% exacto · todos los destinatarios, la wallet
2026-07-20	El mismo swap con recipient = atacante (simulado, sin mover fondos)	DENEGADO revierte antes de ejecutar

Las tres llamadas de ese batch atraviesan cada una un guardián distinto: `USDC.approve(Permit2)` es caso 2b y se comprueba contra `allowedRecipients`; `Permit2.approve` ES CASO 3 con `PIN_PROTOCOL` en el spender; `UniversalRouter.execute` ES CASO 3 con `DEEP` y lo decide el validator.

Comportamiento del gate, medido antes y después de la activación del validator (48 h):

REGISTRY.VALIDATE(. . .)	ANTES DE ACTIVAR	DESPUÉS DE ACTIVAR
Swap legítimo (recipient = wallet)	false	true
Intento de robo (recipient = atacante)	false	false
Comando 0x40 (puente cross-chain)	false	false
Destino ≠ el router al que está atado	false	false

La corrección de la máscara queda por tanto confirmada en mainnet, no solo en tests, y la propiedad fail-closed se ve en la columna «antes»: hasta que el validator no estuvo activo, incluso el swap legítimo se denegaba.

Gas

OPERACIÓN	GAS
createWallet	4.548.977
authorizeAgent	166.803
setCallPolicy	25.912
executeAsAgent (caso 3, con policy comprobada)	64.758

Despliegue V3 — CREATE2 determinista

CONTRATO	DIRECCIÓN (IDÉNTICA EN LAS SEIS REDES)
SmartWalletFactory V3	0xD42F61AA856A4f47885Ecd2D0ce119411d53C192
AgentWalletFactory V3	0xd866a7563cDaC9F71423be3332b62c329C676064
ValidatorRegistry	0x5e371D54AC97a57B0a99145Ed04A3c9fA07850C2
HookRegistry	0x551C6e7ABdA04a110790888e711198f25621b066

Redes: Arbitrum One (42161), Ethereum (1), BNB Chain (56), Base (8453), Polygon (137) y Arbitrum Sepolia (421614). Los cuatro contratos se re-comprobaron el 2026-07-27 vía la API de los exploradores: presentes, byte a byte idénticos y con código fuente verificado en las seis cadenas.

Los validators del Universal Router están atados a un router cada uno, así que su dirección cambia por red:

RED	VALIDATOR DEL UR	ESTADO EN EL REGISTRY
Arbitrum One	0xD1522594e185C882bfDEc6FFD4DE8a53F69AF0Ca	Activo

Ethereum	0x3615a0Fc0663C0201B543deC9816d5Ef30a82ffb	Activo
BNB Chain	0x6945f861e0D7FCD566739000f3fDC1D44896D815	Activo
Base	0x4A2Fc73AAdEfC84513defaa0c444d2150AC6A6D8	Activo
Polygon	0xC1C584E4149eD2EFfe20DA0155b1B8257232102fF	Activo
Arbitrum Sepolia	0x87550034627966e32d82E3b8AdB3f19c62E8d86E	Desplegado, sin activar

Estado leído directamente de `validators(router)` en cada cadena el 2026-07-27. Un validator desplegado pero sin activar es inerte en la dirección segura: las llamadas DEEP correspondientes fallan cerradas. Los manifiestos de procedencia por red (direcciones más codehashes del router y del validator) están en `contracts/deployments/`. Los validators del `PositionManager` de Uniswap v4 y el registry de hooks están desplegados en las seis redes pero aún no activados en el registry, y quedan fuera del alcance de este informe.

Build congelada

Unas direcciones CREATE2 deterministas exigen una build congelada: solc **0.8.36**, `via_ir`, `optimizer_runs = 50`, EVM cancan. Ese compilador se eligió deliberadamente para esquivar el bug del pipeline via-IR del rango 0.8.28–0.8.33. El tamaño runtime de la factory de agentes es de **24.283 bytes**, con 293 bytes de margen bajo el límite EIP-170 — cualquier adición futura debe medirse contra ese margen.

Capa off-chain — SDK y MCP 0.2.0

Las librerías cliente se hicieron conscientes de las policies para que el guardián on-chain no se descubra solo al emitir la transacción. `getCapabilities` ahora condiciona un swap a las policies registradas y al registry de validators, no solo al struct de permisos, y devuelve la versión de la wallet junto con las policies que falten; los cuatro errores de V3 se decodifican en mensajes accionables que distinguen «no hay validator activo» (no reintentar) de «el validator rechazó este calldata» (corregible). `setCallPolicy` deliberadamente **no** se expone en el SDK: las policies las escribe el passkey del propietario, nunca la clave de un agente. El módulo de préstamos Aave y sus planners viajan por el mismo catálogo, siempre con el beneficiario anclado a la wallet.

Migración

El bytecode de la wallet V3 difiere del de V2, así que las direcciones CREATE2 también: los usuarios existentes crean una wallet nueva y mueven sus fondos, igual que en la migración V1 → V2, tras lo cual se hace `kill()` de las factories V2. Las capacidades futuras — registrar más validators, añadir presets de protocolos — son neutrales respecto al bytecode y no obligarán a otra migración.

13 Revisión externa — hallazgos adicionales

Se entregó el código de V3 a un revisor independiente, que planteó cuatro cuestiones. Las cuatro se reprodujeron contra el código en vez de darlas por buenas a partir de su descripción; los tests están en `test/ThirdPartyClaims.t.sol`. Las cuatro se sostienen, aunque dos son más estrechas de lo que se enunciaba y una es una propiedad de ERC-20 más que de esta wallet. Ninguna había aparecido en las tres rondas internas.

13.1 Apropiación de guardianes a través de la factory

ALTA

CORREGIDO EN CÓDIGO

PENDIENTE DE DESPLEGAR EN V4

La dirección `CREATE2` de una wallet se deriva únicamente de la clave pública de la passkey; los guardianes no forman parte de ella. `createWallet` no exige autorización alguna y fija los guardianes en la misma llamada, y `setGuardians` es de un solo uso. Quien despliegue una dirección primero elige, de forma permanente, a las tres partes que pueden rotar a su propietario. Si la wallet ya existe, la factory la devuelve sin cambios, así que quien llegue segundo recibe la wallet ocupada sin error y sin ninguna señal.

La apropiación completa se reprodujo de principio a fin

(`test_Claim1_GuardianSquattingTakesOverTheWallet`): un atacante despliega la dirección determinista de la víctima con sus propios guardianes; la app de la víctima «crea» después la wallet y recibe de vuelta esa misma dirección; dos de los guardianes del atacante abren una recuperación, agotan el timelock de 48 horas y rotan el signer. La wallet y su saldo son suyos.

El diseño multichain amplifica esto precisamente por ser determinista: la clave pública de la passkey queda visible on-chain en cuanto el usuario despliega en cualquier red, y la misma dirección es reclamable en todas las demás. La exposición son, por tanto, **las redes donde un usuario aún no ha desplegado**, no las wallets ya en uso — una vez fijados los guardianes correctos no se pueden sustituir. El timelock de 48 horas junto con `cancelRecovery` es una defensa real, pero solo en una red que el propietario esté vigilando, lo que por definición excluye aquellas a las que no ha desplegado.

Una parte de la recomendación del revisor no se sostiene. Rechazar guardianes duplicados on-chain es higiene, no un agujero: las aprobaciones se llevan por dirección, así que un guardián duplicado no puede aprobar dos veces ni alcanzar por sí solo el umbral de 2 de 3 (`test_Claim1_DuplicateGuardianCannotSelfApprove`). Los duplicados degradan el esquema a 2 de 2; no permiten una apropiación en solitario.

Corrección, ya aplicada en código. La factory ya no elige guardianes: despliega la wallet sin ninguno, y `setGuardians` queda protegida con `msg.sender == address(this)` —el mismo patrón de auto-llamada que ya usan `authorizeAgent` y `setCallPolicy`— de modo que solo una operación firmada con la passkey puede fijarlos. Los duplicados también se rechazan ahora. El ocupante se queda con un cascarón vacío que no puede configurar y, de hecho, ha pagado el gas del despliegue de la víctima. Los guardianes no se leen en ningún sitio salvo en la ruta de recuperación, así que una wallet sin ellos opera con normalidad; lo único no disponible es la recuperación hasta que el dueño firme. Lo cubren siete tests de regresión, incluidos la vía real del propietario —un batch de `execute()` cuyo item apunta a la propia wallet— y el hecho de que un agente autorizado tampoco llega, porque `executeAsAgent` rechaza cualquier item dirigido a la wallet.

Al corregirlo aparecieron dos hallazgos emparentados. El `credentialId` viajaba por la factory como campo de evento sin autenticar, así que un ocupante podía publicar uno falso para la dirección de otro. No puede costar fondos —las firmas se validan contra la clave pública guardada— pero un

cliente que se fíe preselecciona una credencial que el dueño no tiene, y el prompt de la passkey falla. Se ha sacado de la factory por completo: ahora lo anuncia la propia wallet, en la misma llamada firmada que fija los guardianes, mediante `CredentialSet(bytes32 indexed credentialHash, bytes credentialId)`. El hash va indexado para que un cliente con el `rawId` de una passkey pueda encontrar la wallet a la que pertenece. Aparte, `executeRecovery` rota el signer pero dejaba la credencial intacta, de modo que tras una recuperación por guardianes la credencial on-chain apuntaba a una passkey que el dueño ya no tenía; `setCredentialId`, también con auto-llamada, lo cierra.

Disponible hoy, sin cambiar contratos: leer `guardians(0..2)` en una red antes de tratar una wallet como propia, y desplegar de forma proactiva en todas las redes soportadas para no dejar ninguna dirección sin reclamar.

13.2 Los presupuestos de token no cubren el gasto de caso 3

ALTA — condicionada a que exista una allowance previa

ABIERTO — mitigaciones abajo

Los contadores por token solo acumulan para los selectores `transfer` y `approve` de ERC-20: a `_checkTokenWhitelistAndAmount` se llega desde esas dos ramas y desde ninguna más. Una llamada de caso 3 gasta por tanto tokens sin tocar `tokenMaxAmounts`, el límite diario ni el presupuesto vitalicio — y sin comparar el activo que viaja dentro de su calldata con `allowedTokens`. Esto es deliberado hasta cierto punto: el modelo cobra el presupuesto en el paso del `approve`, que es lo que afirma la batería de Aave sobre fork, así que en el flujo habitual de aprobar y luego operar el presupuesto sí se consume.

El hueco son las **allowances previas**, y el caso común no es rebuscado: un propietario que haya concedido una allowance amplia a un router o a Permit2 desde la interfaz —práctica estándar— deja al agente moviendo ese token por la vía de caso 3 indefinidamente sin consumir una unidad de su presupuesto. Reproducido en `test_Claim2_PreExistingAllowanceEscapesTokenBudgets`: con los tres toques de token puestos a 10, el agente mueve 500 a través de un router whitelisteado y ambos contadores se quedan a cero. El propio `approve(address,address,uint160,uint48)` de Permit2 es asimismo un selector distinto y tampoco cuenta.

Corrección de severidad. Un borrador anterior de esta sección describía la pérdida como operar por encima del volumen previsto, apoyándose en que las call policies anclan todos los destinatarios a la wallet. Eso se queda corto. El anclaje garantiza *dónde aterriza el output*, no *cuánto vale*: un agente comprometido puede cambiar un activo valioso por uno sin valor a través de un pool que él controla, y el token inútil vuelve obedientemente a casa mientras el valor se queda en la liquidez del atacante. Nada en el diseño actual se opone — el validator del Universal Router no contiene ni una sola referencia a cantidades o precios, y las policies de `SwapRouter02` anclan el destinatario y nada más, así que un swap con `amountOutMinimum = 1` pasa todos los controles. Donde exista una allowance viva, esto es un vaciado completo de los tokens que cubra, no deslizamiento de precio.

La selección de tokens lo estrecha, pero no lo cierra. Elegir qué tokens puede tocar un agente sí gobierna `transfer` y `approve`, de modo que un token no seleccionado y sin allowance viva no se puede mover: el agente tampoco puede aprobarlo. Quedan dos huecos. Un token que ya arrastra una allowance es alcanzable al margen de la selección. Y el `borrow` y el `withdraw` de Aave no consumen allowance alguna, así que ahí no hay nada que poner a cero: un agente con policy de Aave puede endeudar la wallet en un activo que el propietario nunca seleccionó, contra su propio colateral, con el riesgo de liquidación que eso implica. Los fondos aterrizan en la wallet —es daño, no robo— pero es justo el daño que la selección de tokens debería haber evitado.

Plan de remediación

La contabilidad completa dentro de la wallet exigiría que los validators profundos decodifiquen y devuelvan los pares (token, gasto máximo) que ven, acumulándolos la wallet por `_applyTokenSpend` y contando `amountInMaximum` en las rutas de salida exacta. Eso toca todos los validators más la wallet, y la factory de agentes tiene 101 bytes de margen bajo EIP-170. Por eso se escalona en vez de intentarse ahora.

FASE	MEDIDA	COSTE
Ahora solo interfaz	Renombrar el límite para que deje de sugerir un tope global — gobierna transferencias y autorizaciones directas, no el volumen DeFi	Ninguno
	Al autorizar, leer las allowances de cada token relevante hacia cada protocolo que las policies del agente puedan alcanzar, y exigir una revocación firmada de las que no sean cero	Frontend + un batch firmado
	Ejecutar las operaciones del agente como batches atómicos: <code>approve(exacto) → llamada al protocolo → approve(0)</code> , de modo que la primera pata consuma presupuesto y nada sobreviva al batch	Frontend; funciona con el contrato actual
	Para Permit2, comprobar las dos capas —la allowance ERC-20 hacia Permit2 y la allowance interna de Permit2 hacia el router— con cantidades exactas y expiraciones cortas	Frontend
	Una pantalla de emergencia para el dueño que revoque todas las allowances, y saldos moderados en las wallets que lleven agentes	Frontend + operativa
Después capa de validators	Control de precio en los validators de swap: rechazar una ruta cuyo <code>amountOutMinimum</code> sea inverosímil frente a un oráculo o un TWAP. Un validator es una función <code>view</code> , así que puede consultarlo. Esta es la medida que cierra el vaciado por pool amañado.	Validators nuevos; 48 h de gobernanza por red
	Un validator de Aave que compruebe el argumento <code>asset</code> contra los tokens seleccionados de la wallet, para que la selección gobierne también el préstamo	Validator nuevo; mismo ciclo
Más adelante	Un vault de agente por el que el agente esté obligado a pasar, de modo que la cantidad entregada sea un techo físico y los routers salgan de <code>allowedProtocols</code>	Contrato nuevo + integración
	Contabilidad dentro de la wallet, cuando un rediseño de la factory libere el bytecode necesario	Trabajo de escala V5

Cómo baja la severidad. La calificación viene de dos cosas: la pérdida no tiene techo, y la condición previa es comportamiento normal del usuario. Fíjese en lo que ocurre cuando no existe ninguna allowance viva — el agente tiene que hacer `approve` antes de poder gastar, y ese `approve` sí se cuenta contra el presupuesto por token y pasa por las listas de tokens y destinatarios. El presupuesto vuelve a acotarlo todo. El hallazgo no es por tanto «el presupuesto no funciona» sino «el presupuesto no funciona *mientras exista una allowance viva*», y esa condición es atacable sin tocar los contratos.

ESTADO	QUÉ QUEDA ACOTADO	SEVERIDAD
Hoy	Nada, si existe una allowance	ALTA
+ higiene de allowances (a cero al autorizar, approvals exactos y atómicos, expiraciones cortas en Permit2)	El gasto acumulado, vía el presupuesto de token	MEDIA
+ control de precio en los validators de swap	También la pérdida por operación	BAJA

La higiene sola se queda en media porque es una foto: el dueño puede conceder una allowance desde cualquier dApp después. Baja la probabilidad; solo el validator baja el impacto. Dos salvedades: el borrow y el withdraw de Aave no consumen allowance, así que la higiene no llega ahí hasta que un validator de Aave compruebe el asset; y un límite de precio acota cada operación, no el total — acotar el gasto acumulado de caso 3 exige la contabilidad en la wallet que el bytecode no puede albergar hoy.

Lo que las medidas de interfaz no pueden hacer. Impedir approvals ilimitados mientras haya un agente activo no es aplicable desde el frontend de la propia wallet: el propietario puede conceder una allowance desde cualquier dApp, y nada on-chain lo impide. La comprobación en el momento de autorizar es por tanto una foto que cierra el caso común —la allowance olvidada— no un invariante. Solo la capa de validators lo convierte en uno.

13.3 Pausar o revocar un agente no retira sus allowances

BAJA

INHERENTE A ERC-20

Una allowance vive en el contrato del token, así que nada de lo que haga la wallet puede retirarla. Reproducido en `test_Claim3_AllowanceSurvivesPauseAndRevoke`: el agente aprueba dentro de su tope, el propietario pausa y revoca, y el spender se lleva igualmente el importe completo. El revisor señala correctamente que el importe se cargó al presupuesto en el momento de aprobar, así que no se elude ningún límite. Es una propiedad de cualquier smart account, no un defecto de esta, y afecta a la respuesta ante incidentes más que al modelo de permisos: tras revocar un agente, el propietario debería además poner a cero las allowances que le concedió. Un aviso explícito al pausar y una pantalla de revocación de allowances lo cubren.

13.4 approve puede colar ETH saltándose la lista de destinatarios

BAJA

CORREGIDO EN CÓDIGO

Una ejecución que lleva a la vez un `value` positivo y `calldata` de `approve` la clasifica el validator del agente como operación de token —de modo que la comprobación de destinatario se aplica al `spender`— mientras que el `execute()` padre cae a caso 3 y reenvía el ETH a la dirección del token. Los topes de ETH siguen aplicándose, porque el `value` se acumula incondicionalmente; lo que se salta es la lista blanca de destinos. Reproducido en `test_Claim4_ApproveCarriesEthPastRecipientWhitelist`, donde un envío de ETH normal a esa misma dirección revierte con `RecipientNotAllowed` y el mismo ETH montado sobre un `approve` llega.

La severidad queda limitada por un detalle que el revisor no menciona: el `approve` del destino tiene que ser payable para que la llamada llegue a funcionar. Los ERC-20 estándar no lo son, así que el intento simplemente revierte; el test necesita un token payable fabricado a propósito para demostrar la vía. Explotarlo exige por tanto que el propietario haya whitelisteado un token inusual u hostil. Al lado hay una desprolijidad emparentada: un `transfer` que lleve `value` lo ejecuta el padre sin reenviar el ETH, y aun así el agente carga ese valor a su presupuesto de ETH. Ambas quedan ahora rechazadas: una llamada de token que lleve valor revierte con `TokenCallWithValue`, verificado por un test de regresión que además confirma que un `approve` normal, sin valor, sigue funcionando.

14 Batería adversarial — recuperación y firma

La capa de permisos se había atacado desde todos los ángulos a lo largo de tres rondas; la recuperación por guardianes y la vía de firma, no. Las dos custodian fondos, así que las dos recibieron el mismo trato: veinte tests escritos para conseguir que funcione algo que no debe (`test/RecoveryAdversarial.t.sol`, `test/SignatureAdversarial.t.sol`).

14.1 Vía de firma — sin hallazgos

Diez ataques contra la vía ERC-7739 / WebAuthn, los diez rechazados. Los dos que más importan son los de replay, porque el diseño de esta wallet invita a ambos: la dirección es idéntica en todas las redes, y una misma passkey respalda una smart wallet y una agent wallet en dos direcciones distintas.

ATAQUE	RESULTADO
Reutilizar en otra red una firma hecha para la misma dirección	Rechazado el chainId va atado en el dominio del verificador
Reutilizar una firma entre la smart wallet y la agent wallet del mismo dueño	Rechazado el contrato verificador va atado
Firma maleada (s sustituida por N - s)	Rechazado
Firma a cero	Rechazado
Firma sobre otro contenido, o sobre el dominio de otra dApp	Rechazado
authenticatorData manipulado (quitando el flag de verificación de usuario)	Rechazado
Challenge cambiado por otro digest	Rechazado
webauthn.create colado donde se exige webauthn.get	Rechazado

14.2 Recuperación — tres hallazgos

La recuperación no revocaba los agentes. **CORREGIDO EN CÓDIGO** `executeRecovery` rotaba el signer y no tocaba nada más, así que un agente autorizado antes de la recuperación seguía gastando después, con su presupuesto intacto. Importa porque el compromiso es justamente el motivo por el que un dueño recurre a la recuperación: quien recuperaba creyendo haber echado al atacante no le había revocado el agente. Ahora la recuperación pausa los agentes mediante un hook `_afterRecovery`. Pausar en vez de revocar es deliberado: gas constante sea cual sea la lista de agentes, reversible, y conserva la configuración de cada uno para que el nuevo dueño la juzgue en vez de tener que reconstruirla. La guarda es `if (!paused()) _pause()`: sin ella, una wallet con los agentes ya pausados no habría podido completar una recuperación.

Los guardianes no se podían rotar nunca. **CORREGIDO EN CÓDIGO** `setGuardians` era de un solo uso, así que un guardián cuya clave se perdiera o se comprometiera después era permanente durante toda la vida de la wallet, y el único remedio era crear otra wallet y mover los fondos. El conjunto es ahora reemplazable por el dueño. Sigue siendo una auto-llamada, de modo que la apropiación del §13.1 continúa cerrada, y la rotación se rechaza con una recuperación en vuelo (`RecoveryActive`) — si no, una passkey robada podría cambiar los guardianes por debajo de una recuperación ya en marcha. El dueño cancela primero y rota después.

Un solo guardián puede bloquear una recuperación indefinidamente.

ABIERTO — ACEPTADO

`initiateRecovery` se puede llamar siempre que haya menos de dos aprobaciones, y sobrescribe la clave pendiente y pone el `timelock` a cero. Un guardián malicioso solo tiene por tanto que adelantarse a cada segunda aprobación para que la pareja honesta no alcance nunca el umbral. Es denegación de servicio contra el mecanismo de rescate, no robo: el dueño no llega a rotar. Y es una carrera, no un candado — un test acompañante demuestra que en cuanto la pareja honesta mete dos aprobaciones, ya nadie puede reiniciarla. Aceptado como limitación documentada.

El modelo de confianza, sin adornos. Dos guardianes coludidos se quedan con la wallet si el dueño no cancela dentro de la ventana de 48 horas — eso es lo que hace que la recuperación funcione, y así está probado. Implica que el dueño debe vigilar todas las redes donde exista su wallet, que es el mismo requisito operativo que la apropiación de guardianes del §13.1 dejó al descubierto por otra vía.

15 V4 — Remediación pendiente

Siete hallazgos exigían cambiar el bytecode y están corregidos en código, a la espera de un despliegue. Como las direcciones CREATE2 se derivan del bytecode, no se pueden aplicar a las wallets ya desplegadas: la migración es el mismo playbook que V1 → V2 y V2 → V3 —los usuarios recrean su wallet y mueven fondos, tras lo cual se hace `kill()` de las factories sustituidas—. Viajan en un solo despliegue en vez de forzar varias migraciones. Los contratos se renombraron a V4 y el dominio EIP-712 va detrás (`BVCCSmartWalletV4`), igual que en la generación V2 → V3.

ELEMENTO	ESTADO
Reentrada cruzada (§12.1)	Corregido dos capas, cada una con su test de regresión
Apropiación de guardianes (§13.1)	Corregido la factory ya no los elige; <code>setGuardians</code> exige <code>passkey</code>
<code>credentialId</code> sin autenticar (§13.1)	Corregido fuera de la factory, a un evento firmado que emite la wallet
Credencial obsoleta tras recuperación (§13.1)	Corregido <code>setCredentialId</code> , con auto-llamada
<code>approve</code> con ETH (§13.4)	Corregido las llamadas de token deben llevar valor cero
Los agentes sobreviven a la recuperación (§14.2)	Corregido la recuperación pausa los agentes
Los guardianes no se podían rotar (§14.2)	Corregido el dueño puede reemplazarlos, nunca durante una recuperación
Contabilidad de token en caso 3 (§13.2)	Abierto escalonado; la fase de <code>validators</code> es la que lo cierra
Bloqueo de recuperación por un guardián (§14.2)	Abierto aceptado; denegación de servicio, no robo
Zero-check en constructores de <code>validators</code> (§12)	Abierto aceptado; inerte, y aplazado para viajar con el próximo despliegue de <code>validators</code>
UX de revocar allowances y defaults de topes (§13.2)	Abierto trabajo de interfaz — deliberadamente fuera del contrato

Objetivos de despliegue de V4

CONTRATO	DIRECCIÓN (CREATE2, IDÉNTICA EN TODAS LAS REDES)
SmartWalletFactory V4 (salt ...0A)	0xfdd105197109244483b5f870501326E6faec9F93c
AgentWalletFactory V4 (salt ...0B)	0xf3A61F9d64d45362E149A111289546523BCd26a6
ValidatorRegistry	0x5e371D54AC97a57B0a99145Ed04A3c9fA07850C2 — sin cambios

El registry queda deliberadamente intacto: es la dirección que va compilada como constante dentro de cada wallet, y el test de congelación demuestra que la wallet V4 sigue resolviendo a ella. Los `validators` se comparten entre generaciones, así que no hay que volver a registrar nada para V4.

La factory de agentes queda en **24.541 bytes**, 35 por debajo del límite EIP-170, con `executeAsAgent` en caso 3 a 64.972 gas. Ese margen es estrecho, y es la razón de que dos medidas de endurecimiento candidatas se dejaran deliberadamente fuera del contrato: exigir topes de token no

nulos cuando un agente tiene acceso a protocolos, y limitar las aprobaciones del propio dueño mientras haya un agente activo. Las dos pertenecen a la interfaz, donde no cuestan nada y pueden explicarse. El bytecode restante queda reservado para medidas sin equivalente fuera de la cadena.

Hasta que V4 esté vivo, cuatro mitigaciones no requieren tocar contratos: autorizar a los agentes con una lista de destinatarios no vacía que excluya su propia dirección; verificar los guardianes de una wallet en una red antes de darla por tuya o meterle fondos; poner a cero cualquier allowance viva hacia los protocolos que un agente pueda alcanzar antes de autorizarlo; y desplegar proactivamente en todas las redes soportadas para no dejar ninguna dirección sin reclamar.

Batería de tests a fecha de redacción: **303 tests, todos pasan**, incluido el guard de congelación de CREATE2 ahora que las constantes de V4 están puestas.

16 Conclusión

La BVCC Agent Wallet aplica un modelo de permisos completo y por capas. Se identificaron tres incidencias de severidad alta a lo largo de tres rondas de revisión, cada una reproducida, remediada, rediseñada y re-verificada, y la batería de permisos on-chain pasa sobre el bytecode parcheado. Al preparar esta edición aparecieron cuatro más: una reentrada cruzada, la apropiación de guardianes por la factory, el gasto sin acotar del caso 3 (§13.2) y los agentes sobreviviendo a una recuperación. Todas menos una están ya corregidas en código y esperan una generación V4 de factories, junto con los arreglos de la credencial, la rotación de guardianes y el approve con valor. La recuperación por guardianes se ejecutó y verificó de extremo a extremo — rotación de propiedad a una nueva passkey y rechazo on-chain de la clave antigua (ver §8). La ronda V2 corrigió un problema de gas en mainnet en el snapshot de comisiones y concluyó con un despliegue multichain determinista (ver §9).

La ronda V3 es el cambio más profundo hasta ahora en la vía del agente. Cerró un vector de exfiltración que los presupuestos no podían ver, haciendo las llamadas DeFi del agente default-deny por selector y anclando el destino o bien a la propia wallet o bien a un validator on-chain cuyo registro es asimétrico: inmediato para denegar, 48 horas para permitir (ver §10 y §11). Dentro de ese trabajo se cazaron dos bypasses en potencia: una máscara de comandos que habría aceptado un depósito en un puente cross-chain como si fuera un swap, y una vía `increaseLiquidity` sin comprobación de propiedad, que se dejó deliberadamente sin registrar. El modelo está verificado por 264 tests unitarios, de fork y de fuzz, por un swap real en mainnet en el que todos los destinatarios resuelven a la wallet y la comisión es exacta, y por el rechazo de la variante adversarial de ese mismo swap antes de ejecutarse (ver §12).

El riesgo residual se expone sin adornos. Las llamadas a protocolo (caso 3) son seguras en el sentido que V3 se propuso: el destino queda anclado. Pero anclar el destino no es lo mismo que acotar el valor, y donde exista una allowance viva esa diferencia es la pérdida entera. Hasta que salga V4, una clave de agente comprometida sobre una wallet dejada en el default permisivo vale más que su presupuesto, y una dirección en una red donde el propietario nunca ha desplegado la puede reclamar otro. El plan escalonado del §13.2 dice qué puede estrechar la interfaz ahora y qué necesita la capa de validators para cerrarse de verdad.

Junto a los hallazgos conviene anotar tres lecciones. La reentrada se descartó una vez, con un razonamiento plausible sobre EOAs, hasta que una prueba de concepto lo contradijo: las suposiciones sobre la forma de una dirección merecen un test, no un argumento. Los problemas más graves de aquí aparecieron al volver a mirar código que ya había pasado una revisión — la apropiación de guardianes vino de un revisor externo, la rotación de credencial de una segunda pasada de ese mismo revisor, y dos más de atacar la recuperación de frente. Y la cifra del análisis estático se movió en uno durante todo el proceso: no subió mientras la reentrada fue explotable durante tres generaciones, ni bajó cuando el arreglo la cerró. Las herramientas encuentran patrones; los hallazgos de aquí salieron de leer el código y preguntarse qué haría un atacante con él. Los contratos siguen siendo de revisión interna y no están auditados externamente.

Anexo — identificadores clave

ELEMENTO	VALOR
Red	Arbitrum Sepolia (421614)
Wallet parcheada	0x1ED261BA084c4E6117daf289f3ca233BBA3cF0C3

Agente (rol B)	0x38529C66F3cf22453D66B9E2A20FdF2676544aB4
AgentWalletFactory (Sepolia, pre-V2)	0xc87aa10747A92B472EF6B36e190B84c897a2953e
SmartWalletFactory (Sepolia, pre-V2)	0xa5290A51a73903176e09C864E1542a07da67BD12
SmartWalletFactory V2 (todas las redes)	0x230b7010529AB6977Dd8581B3eF018ef865BdEf1
AgentWalletFactory V2 (todas las redes)	0x8D9e24022777173AD6336e00884b6C87c7EF054c
SmartWalletFactory V3 (todas las redes)	0xD42F61AA856A4f47885Ecd2D0ce119411d53C192
AgentWalletFactory V3 (todas las redes)	0xd866a7563cDaC9F71423be3332b62c329C676064
ValidatorRegistry V3 (todas las redes)	0x5e371D54AC97a57B0a99145Ed04A3c9fA07850C2
HookRegistry V3 (todas las redes)	0x551C6e7ABdA04a110790888e711198f25621b066
Wallet de pruebas V3 (Arbitrum One)	0x605A4C65dBa64bd26d9F7701e1e4Cda48c594A5c
Admin de gobernanza (hardware wallet)	0x3145Bd5e2489d8bDdAb17F23F26F07Ac5aD55A4c
Redes V2/V3	Arbitrum One · Base · BNB Chain · Ethereum · Polygon · Arbitrum Sepolia
Build V3	solc 0.8.36 · via-IR · optimizer 50 · EVM cancan (congelada para CREATE2)
Herramientas	Slither 0.11.3 · Foundry (forge 1.5.1) · viem 2.x

Confidencial — Block Venture Chain Capital. Preparado para revisión interna e inclusión en el repositorio. La batería de permisos V1 se ejecutó en una testnet pública (Arbitrum Sepolia); la capa de call policies de V3 se verificó además en Arbitrum One mainnet. Las factories V2 y V3 están desplegadas en Arbitrum One, Ethereum, BNB Chain, Base, Polygon y Arbitrum Sepolia.